第一章作业

第1章

1.7

[20] <1.6> 同一个指令集体系结构有两种不同的实现方式。根据CPI的不同将指令分成4类（A、B、C和D），P1的时钟频率为2.5GHz，CPI分别为1、2、3和3；P2时钟频率为3GHz，CPI分别为2、2、2和2。

给定一个程序，有 1.0×10^6 条动态指令，按如下比例分为4类：A，10%；B，20%；C，50%；D，20%。那么P1和P2哪个更快?

a. 每种实现方式总的CPI是多少?

$$\mathrm{CPI}_{P1} = 0.1 \times 1 + 0.2 \times 2 + 0.5 \times 3 + 0.2 \times 3 = 0.1 + 0.4 + 1.5 + 0.6 = 2.6$$

$$\mathrm{CPI}_{P2} = 0.1 \times 2 + 0.2 \times 2 + 0.5 \times 2 + 0.2 \times 2 = 2.0$$

b.计算两种情况下的时钟周期数。

1.8

[15] <1.6> 编译器对应用的性能有较深的影响。假定一个程序，如果采用编译器A，则动态指令数为1.0E9，执行时间为1.1s；如果采用编译器B，则动态指令数为1.2E9，执行时间为1.5s。

a. 在给定处理器时钟周期为1ns时，求每个程序的平均CPI。

时钟周期 $T_c = 1\ \mathrm{ns}$，即频率 $f = 1\ \mathrm{GHz}$。
编译器 A：指令数 1.0×10^9，执行时间 $1.1\ \mathrm{s}$

$$\mathrm{CPI}_A = \frac{T_A \times f}{\text{指令数}} = \frac{1.1 \times 1 \times 10^9}{1.0 \times 10^9} = 1.1$$

编译器 B：指令数 1.2×10^9，执行时间 $1.5\ \mathrm{s}$

$$\mathrm{CPI}_B = \frac{1.5 \times 1 \times 10^9}{1.2 \times 10^9} = 1.25$$

b. 假定编译器在两个不同的处理器上运行。如果这两个处理器的执行时间相同，求运行编译器A产生的代码的处理器时钟，相对于运行编译器B产生的代码的处理器的时钟快多少?

设两个处理器执行时间相同：

$$\frac{1.0 \times 10^9 \times 1.1}{f_A} = \frac{1.2 \times 10^9 \times 1.25}{f_B}$$

$$\frac{1.1}{f_A} = \frac{1.5}{f_B} \quad \Rightarrow \quad f_A = \frac{1.1}{1.5} f_B \approx 0.733 f_B$$

运行 A 代码的处理器时钟频率是运行 B 代码处理器的 73.3%，或者说 B 的时钟比 A 快 $1/0.733 \approx 1.364$ 倍。

c. 假设开发了一种新的编译器，只用6.0E8条指令，平均CPI为1.1。求使用新编译器相对于使用编译器A和B的加速比。

新编译器：指令数 6.0×10^8，CPI = 1.1，运行在相同处理器（频率 f）。
时间：$T_{\text{new}} = \frac{6.0\times10^8 \times 1.1}{f} = \frac{6.6\times10^8}{f}$
对 A：$T_A = \frac{1.0\times10^9 \times 1.1}{f} = \frac{1.1\times10^9}{f}$
加速比 $S_A = \frac{T_A}{T_{\text{new}}} = \frac{1.1\times10^9}{6.6\times10^8} = 1.667$
对 B：$T_B = \frac{1.2\times10^9 \times 1.25}{f} = \frac{1.5\times10^9}{f}$
加速比 $S_B = \frac{1.5\times10^9}{6.6\times10^8} \approx 2.273$

1.10

在一个处理器中，假定算术指令、load/store指令和分支指令的CPI分别是1、12和5。另外假定一个程序在单个处理器核上运行时需要执行2.56E9条算术指令、1.28E9条load/store指令和2.56E8条分支指令，并假定处理器的时钟频率为2GHz。现假定程序并行运行在多核上，分配到每个处理器核上运行的算术指令和load/store指令数为单核情况下相应指令数除以 $0.7 \times p$（p是处理器的数量），而每个处理器的分支指令的数量保持不变。

1.10.1 **[5] <1.7>** 求出当该程序分别在1、2、4和8个处理器核上的执行时间，并求出每种情况下相对于单核处理器的加速比。

执行时间 $T(p) = \frac{\text{算术数}\times 1 + \text{L/S数}\times 12 + \text{分支数}\times 5}{f}$

p = 1（单核）：
算术 2.56×10^9，L/S 1.28×10^9，分支 2.56×10^8
总周期 = $2.56 \times 10^9 + 1.28 \times 10^9 \times 12 + 2.56 \times 10^8 \times 5$
$= 2.56 \times 10^9 + 15.36 \times 10^9 + 1.28 \times 10^9 = 19.2 \times 10^9$
$T(1) = 19.2 \times 10^9 / (2 \times 10^9) = 9.6\ \text{s}$

p = 2：除数 $0.7 \times 2 = 1.4$
算术 $2.56 \times 10^9 / 1.4 \approx 1.82857 \times 10^9$
L/S $1.28 \times 10^9 / 1.4 \approx 0.914286 \times 10^9$
分支 2.56×10^8
总周期 = $1.82857 \times 10^9 + 0.914286 \times 10^9 \times 12 + 0.256 \times 10^9 \times 5$
$= 1.82857 \times 10^9 + 10.97143 \times 10^9 + 1.28 \times 10^9 = 14.08 \times 10^9$
$T(2) = 14.08 \times 10^9 / (2 \times 10^9) = 7.04\ \text{s}$
加速比 $S(2) = 9.6 / 7.04 \approx 1.364$

p = 4：除数 $0.7 \times 4 = 2.8$
算术 $2.56 \times 10^9 / 2.8 \approx 0.914286 \times 10^9$
L/S $1.28 \times 10^9 / 2.8 \approx 0.457143 \times 10^9$
分支 2.56×10^8
总周期 = $0.914286 \times 10^9 + 0.457143 \times 10^9 \times 12 + 1.28 \times 10^9$
$= 0.914286 \times 10^9 + 5.485716 \times 10^9 + 1.28 \times 10^9 = 7.68 \times 10^9$
$T(4) = 7.68 \times 10^9 / (2 \times 10^9) = 3.84\ \text{s}$
加速比 $S(4) = 9.6 / 3.84 = 2.5$

p = 8：除数 $0.7 \times 8 = 5.6$
算术 $2.56 \times 10^9 / 5.6 \approx 0.457143 \times 10^9$
L/S $1.28 \times 10^9 / 5.6 \approx 0.228571 \times 10^9$
分支 2.56×10^8
总周期 = $0.457143 \times 10^9 + 0.228571 \times 10^9 \times 12 + 1.28 \times 10^9$
$= 0.457143 \times 10^9 + 2.742852 \times 10^9 + 1.28 \times 10^9 = 4.48 \times 10^9$
$T(8) = 4.48 \times 10^9 / (2 \times 10^9) = 2.24\ \text{s}$
加速比 $S(8) = 9.6 / 2.24 \approx 4.286$

1.10.2 **[10] <1.6,1.8>** 如果算术指令的CPI翻倍，那么在处理器核数分别为1、2、4和8个时，对程序的执行时间有何影响?

算术 CPI 变为 2。重新计算各核总周期（仅算术部分乘 2，其他不变）。
p = 1：总周期 = $2.56 \times 10^9 \times 2 + 15.36 \times 10^9 + 1.28 \times 10^9 = 5.12 \times 10^9 + 15.36 \times 10^9 +$

$1.28 \times 10^9 = 21.76 \times 10^9$，$T = 10.88\ \text{s}$
p = 2：算术周期 = $1.82857 \times 10^9 \times 2 = 3.65714 \times 10^9$，L/S 10.97143e9，分支 1.28e9，总和 = 15.90857e9，$T = 7.954\ \text{s}$
p = 4：算术周期 = $0.914286 \times 10^9 \times 2 = 1.828572 \times 10^9$，L/S 5.485716e9，分支 1.28e9，总和 = 8.594288e9，$T = 4.297\ \text{s}$
p = 8：算术周期 = $0.457143 \times 10^9 \times 2 = 0.914286 \times 10^9$，L/S 2.742852e9，分支 1.28e9，总和 = 4.937138e9，$T = 2.4686\ \text{s}$
相比于原 CPI=1 的情况，执行时间均增加。

1.10.3 **[10] <1.6,1.8>** 如果要使单核处理器的性能与四核处理器相当，单处理器中load/store指令的CPI应该降低多少？假定四核处理器的CPI保持不变。

四核原执行时间 $T(4) = 3.84\ \text{s}$。
单核执行时间公式：
$T_{\text{single}} = \frac{2.56\times10^9\times1+1.28\times10^9\times\text{CPI}_{LS}+2.56\times10^8\times5}{2\times10^9}$
令 $T_{\text{single}} = 3.84$，解 CPI_{LS}：
$3.84 \times 2 \times 10^9 = 2.56 \times 10^9 + 1.28 \times 10^9 \cdot \text{CPI}_{LS} + 1.28 \times 10^9$
$7.68 \times 10^9 = (2.56 + 1.28) \times 10^9 + 1.28 \times 10^9 \cdot \text{CPI}_{LS}$
$7.68 = 3.84 + 1.28 \cdot \text{CPI}_{LS}$
$1.28 \cdot \text{CPI}_{LS} = 3.84 \Rightarrow \text{CPI}_{LS} = 3$
原 CPI 为 12，需降低至 3，即降低 75%。

1.14

1.14节提到的另一个陷阱，是通过只改进计算机的一个方面来改进计算机的总体性能。假如一台计算机上运行一个程序需要250s，其中70s用于执行浮点指令，85s用于执行L/S指令，40s用于执行分支指令。

1.14.1 **[5] <1.11>** 如果浮点操作的时间减少20%，总时间将减少多少？

新浮点时间 = $70 \times 0.8 = 56\ \text{s}$
新总时间 = $56 + 85 + 40 + 55 = 236\ \text{s}$
时间减少 = $250 - 236 = 14\ \text{s}$，减少比例 = $14/250 = 5.6\%$

1.14.2 **[5] <1.11>** 如果总时间减少20%，整数操作时间将减少多少？

新总时间 = $250 \times 0.8 = 200\ \text{s}$
设整数操作时间减少 x 秒（原 55 s），其他不变：

$200 = (70 + 85 + 40) + (55 - x) = 195 + 55 - x = 250 - x$
$\Rightarrow x = 50$ s，即整数时间需从 55 s 减少到 5 s，减少 $50/55 \approx 90.9\%$。

1.14.3 **[5] <1.11>** 如果只减少分支指令时间，总时间能否减少20%?

若分支时间减少至 t，其他不变：总时间 = $70 + 85 + t + 55 = 210 + t$。
要求 $210 + t = 200 \Rightarrow t = -10$，不可能。所以不能。

1.15

假定一个程序需要执行50E6条浮点指令、110E6条整数指令、80E6条L/S指令和16E6条分支指令。每种类型指令的CPI分别是1、1、4和2。假定处理器的时钟频率为2GHz。

1.15.1 **[10] <1.11>** 如果要将程序运行速度提高至2倍，浮点指令的CPI需如何改进?

原总周期 = $50 \times 10^6 \times 1 + 110 \times 10^6 \times 1 + 80 \times 10^6 \times 4 + 16 \times 10^6 \times 2$
$= 50 + 110 + 320 + 32 = 512 \times 10^6$ 周期
原时间 $T = 512 \times 10^6 / (2 \times 10^9) = 0.256$ s
目标时间 0.128 s，即目标周期数 = $0.128 \times 2 \times 10^9 = 256 \times 10^6$
设浮点 CPI 变为 x，其他不变：
$50 \times 10^6 \cdot x + 110 \times 10^6 \times 1 + 80 \times 10^6 \times 4 + 16 \times 10^6 \times 2 = 256 \times 10^6$
$50x + 110 + 320 + 32 = 256 \Rightarrow 50x + 462 = 256 \Rightarrow 50x = -206$，不可能。
实际上要求浮点 CPI 为负，说明仅降低浮点 CPI 无法达到速度翻倍。需结合其他优化。

1.15.2 **[10] <1.11>** 如果要将程序运行速度提高至2倍，L/S指令的CPI需如何改进?

设 L/S CPI 变为 y:
$50 \times 1 + 110 \times 1 + 80 \cdot y + 16 \times 2 = 256$
$50 + 110 + 80y + 32 = 192 + 80y = 256 \Rightarrow 80y = 64 \Rightarrow y = 0.8$
L/S CPI 需降至 0.8，但 CPI 不能小于 1，故不可能

1.15.3 **[5] <1.11>** 如果整数和浮点指令的CPI减少40%，L/S和分支指令的CPI减少30%，程序的执行时间能改进多少?

整数和浮点 CPI 减少 40%，即变为 $1 \times 0.6 = 0.6$;
L/S 和分支 CPI 减少 30%，即变为 $4 \times 0.7 = 2.8$，$2 \times 0.7 = 1.4$。
新总周期 = $50 \times 0.6 + 110 \times 0.6 + 80 \times 2.8 + 16 \times 1.4$
$= 30 + 66 + 224 + 22.4 = 342.4 \times 10^6$

原周期 512e6，改进比例 = $1 - 342.4/512 = 1 - 0.66875 = 0.33125$，即执行时间减少 33.125%。

第2章

2.7

[5] <2.2,2.3> 把下面的C代码翻译为MIPS代码。假定变量f、g、h、i和j分别放在寄存器 $\$s0$、$\$s1$、$\$s2$、$\$s3$ 和 $\$s4$。假定数组A和数组B的基地址分别存放在 $\$s6$ 和 $\$s7$ 中。假定数组A和数组B中的元素均为8字节的字：

```
B[8] = A[i] + A[j];
```

```
# 计算 A[i] 的地址：A + i*8
sll $t0, $s3, 3       # $t0 = i * 8
add $t0, $s6, $t0     # $t0 = &A[i]
ld $t1, 0($t0)        # $t1 = A[i]

# 计算 A[j] 的地址：A + j*8
sll $t2, $s4, 3       # $t2 = j * 8
add $t2, $s6, $t2     # $t2 = &A[j]
ld $t3, 0($t2)        # $t3 = A[j]

# 求和
add $t4, $t1, $t3     # $t4 = A[i] + A[j]

# 存储到 B[8]：B + 8*8 = B + 64
addi $t5, $s7, 64     # $t5 = &B[8]
sd $t4, 0($t5)        # B[8] = A[i] + A[j]
```

2.16

假设可以将MIPS寄存器文件扩展到128个寄存器，并将指令集中的指令数扩展为原来的4倍。

2.16.1 **[5] <2.5>** 这将如何影响R型指令的每个位字段的大小?

原 R 型指令格式：
`op` (6 bit) | `rs` (5 bit) | `rt` (5 bit) | `rd` (5 bit) | `shamt` (5 bit) | `funct` (6 bit)

- 寄存器数量从 32 增至 128，需 $\log_2 128 = 7$ 位表示一个寄存器，因此 `rs` 、 `rt` 、 `rd` 各扩展 2 位（从 5 位到 7 位）。
- 指令数扩展 4 倍，即 2^2，需要增加 2 位来区分指令。R 型指令通过 `op` 和 `funct` 共同确定指令，可以将 `op` 扩展 2 位（从 6 位到 8 位），或保持 `op` 不变而扩展 `funct` 。
  典型方案： `op` 扩展 2 位变为 8 位， `rs` 、 `rt` 、 `rd` 各 7 位， `shamt` 保持 5 位， `funct` 保持 6 位，总长度 = 8+7+7+7+5+6 = 40 位。或保持 32 位指令长度，则需压缩其他字段。

2.16.2 **[5] <2.5>** 这将如何影响I型指令的每个位字段的大小?
原 I 型指令格式：
`op` (6 bit) | `rs` (5 bit) | `rt` (5 bit) | `immediate` (16 bit)

- `rs` 和 `rt` 需从 5 位扩展至 7 位。
- 指令数扩展 4 倍，需增加 2 位给 `op` （从 6 位到 8 位）。
- 立即数字段可能需压缩以保持 32 位：新长度 = 8+7+7+imm = 22+imm，若 imm 保持 16 位则总长 38 位；若保持 32 位，则 imm 需减少至 10 位。

2.16.3 **[5] <2.5,2.10>** 在提出的这两种变化中，每种变化如何减少一个MIPS汇编程序的大小？另一方面，如何增大一个MIPS汇编程序的大小?

- **减少大小**：更多寄存器可减少对栈和内存的访问（更多值可保存在寄存器中），从而减少指令数。指令数增加（更多操作码）可能允许更强大的指令，减少指令数。
- **增加大小**：指令编码变长（如果选择 32 位以上）会增大代码尺寸；更多寄存器意味着保存/恢复寄存器的开销可能增加（如函数调用需保存更多寄存器）。

2.17

假设如下寄存器内容：
$\$t0 = 0xAAAAAAAA$，$\$t1 = 0x12345678$

2.17.1 **[5] <2.6>** 对于以上的寄存器内容，执行下面的指令序列后 $\$t2$ 的值是多少?

结果：$\$t2 = 0xBABEFEF8$

2.17.2 **[5] <2.6>** 对于以上的寄存器内容，执行下面的指令序列后 $\$t2$ 的值是多少?

结果：$\$t2 = 0xAAAAAAA0$

2.17.3 **[5] <2.6>** 对于以上的寄存器内容，执行下面的指令序列后 $\$t2$ 的值是多少?

结果：$\$t2 = 0x00005555$

2.22

假设程序计数器（PC）被设置为 $0x20000000$

2.22.1 **[5] <2.10>** 如果使用MIPS的jump-and-link（jal）指令，可以转移到的地址范围是多少？（也就是说，跳转指令执行之后PC值可能的集合是什么？）

`jal` 使用 J 型格式，26 位立即数乘以 4 作为偏移，与 PC 高 4 位拼接形成 32 位地址。
可跳转范围为以当前 PC 所在 256 MB 区域内的任意地址，即 $[0x20000000, 0x21FFFFFF]$（因为 $0x20000000$ 的高 4 位是 0x2，低 28 位可变）。
实际可达地址集合：$\{\mathrm{PC}[31:28] \ll 28 \parallel (\mathrm{imm26} \ll 2)\}$，即 $0x20000000$ 到 $0x21FFFFFF$ 之间每 4 字节对齐的地址。

2.22.2 **[5] <2.10>** 如果使用MIPS的相等则分支（beq）指令，可以转移到的地址范围是多少？（也就是说，分支指令执行之后PC值可能的集合是什么？）
是否可以使用MIPS的跳转（j）指令将PC设置为地址 $0x40000000$？是否可以使用MIPS的相等则分支（beq）指令将PC设置为该地址?

`beq` 使用 I 型格式，16 位立即数乘以 4 作为相对偏移，与当前 PC + 4 相加。
偏移范围为 $-2^{15} \times 4$ 到 $(2^{15} - 1) \times 4$，即 -131072 到 131068 字节。
所以可跳转地址范围为 $[0x20000000 + 4 - 131072, 0x20000000 + 4 + 131068] = [0x1FFE0004, 0x20020004]$ 之间的 4 字节对齐地址。

- `j` 指令（J 型）可以，因为 `j` 使用 26 位立即数，与当前 PC 高 4 位拼接，如果当前 PC 高 4 位为 0x2，则只能跳转到 0x2xxxxxx 区域，无法到 0x4xxxxxx。但若使用 `j` 指令时，目标地址的高 4 位由 PC 当前值决定，所以如果 PC 在 0x20000000， `j` 无法跳转到 0x40000000。但可以使用 `jr` 或 `jalr` 间接跳转。
- `beq` 只能相对跳转，范围有限，无法从 0x20000000 直接跳转到 0x40000000

2.24

考虑如下的 MIPS 循环：

```
LOOP:  slt $t2, $s0, $t1
       beq $t2, $0, DONE
       subi $t1, $t1, 1
       addi $t2, $t2, 2
       j LOOP
DONE:
```

2.24.1 **[5] <2.7>** 假设寄存器 $\$t1$ 的初始值为10，假设 $\$s0$ 初始值为0，则循环执行完毕时寄存器 $\$s0$ 的值是多少?

初始：$\$t1 = 10$，$\$s0 = 0$。
循环条件： `slt $t2, $s0, $t1` → 若 $0 < 10$ 则 $t2 = 1$，否则 0。
$t2 = 1$， `beq $t2, $0, DONE` 不跳转。
`subi $t1, $t1, 1` → $t1 = 9$
`addi $t2, $t2, 2` → $t2 = 3$（但 $t2$ 在循环内未被后续使用，不影响条件）
循环继续，直到 $t1$ 减到 0 或条件不满足。实际上 $s0$ 始终为 0，条件 `slt $t2, $0, $t1` 在 $t1 > 0$ 时均为真，所以循环执行 10 次，$t1$ 从 10 减到 0 时退出。
循环执行完毕后 $\$s0$ 仍为 0（未被修改）。
答案：$\$s0 = 0$

2.24.2 **[5] <2.7>** 对于上面的循环体，写出等价的 C 代码。假定寄存器 $\$s1$、$\$s2$、$\$t1$ 和 $\$t2$ 分别为整数 A、B、i 和 temp。

```c
int A, B, i, temp;  // 对应 $s1, $s2, $t1, $t2
while (i > 0) {
    i = i - 1;
    temp = temp + 2;
}
```

2.24.3 **[5] <2.7>** 假定寄存器 $\$t1$ 的初始值为 N，上面的 MIPS 汇编循环执行了多少条指令?

设初始 $t1 = N$。循环体执行 N 次（每次 $t1$ 减 1，直到 0）。
每次循环执行 5 条指令（slt, beq, subi, addi, j）。
另外，最后一次 beq 跳转到 DONE，不执行循环体。
总指令数 = $N \times 5 + 1$（最后一条 beq 跳转指令也算在内）。
如果 $N = 0$，则不进入循环，只执行 slt 和 beq 两条指令？实际上循环入口 slt 和 beq 总会执行，然后根据条件决定。一般情况：循环执行次数 N，则 slt 和 beq 执行 $N + 1$ 次（最后一次 beq 跳转），循环体内其他指令执行 N 次。所以总指令数 = $(N + 1) \times 2 + N \times 3 = 2N + 2 + 3N = 5N + 2$

。
总指令数为 $5N + 2$。

2.39

假设给定处理器的算术指令的CPI是1，取数/存数指令的CPI是10，分支指令的CPI是3。假设一个程序由5亿条算术指令、3亿条取数/存数指令和1亿条分支指令组成。

2.39.1 **[5] <1.6,2.13>** 假设向指令集中添加了新的、功能更强的算术指令。通过使用这些功能更强大的算术指令平均可以减少程序执行所需要的25%的算术指令，而时钟周期的开销增长了10%。请问这是好的设计选择吗？为什么？

新算术指令减少 25% 的算术指令，即算术指令数变为 $5 \times 10^8 \times 0.75 = 3.75 \times 10^8$。
时钟周期开销增长 10%，即新周期时间 $T_{\text{new}} = 1.1T_{\text{old}}$，频率降低。
新总周期数 = $3.75 \times 10^8 \times 1 + 3 \times 10^8 \times 10 + 1 \times 10^8 \times 3 = 3.75 \times 10^8 + 30 \times 10^8 + 3 \times 10^8 = 36.75 \times 10^8$ 周期。
新执行时间 = $36.75 \times 10^8 \times 1.1 \times T_{\text{cycle,old}}$，原执行时间 = $38 \times 10^8 \times T_{\text{cycle,old}}$。
加速比 = $\frac{38}{36.75 \times 1.1} = \frac{38}{40.425} \approx 0.94$，性能下降约 6%，不是好设计。

2.39.2 **[5] <1.6,2.13>** 假设可以找到一种可以使算术指令性能达到原来两倍的方法。请问机器的整体加速是多少？假设可以找到一种可以使算术指令性能达到原来10倍的方法，那么机器的性能整体加速又是多少？

算术指令性能提高 2 倍，即 CPI 变为 0.5。
新总周期 = $5 \times 10^8 \times 0.5 + 3 \times 10^8 \times 10 + 1 \times 10^8 \times 3 = 2.5 \times 10^8 + 30 \times 10^8 + 3 \times 10^8 = 35.5 \times 10^8$
加速比 = $38/35.5 \approx 1.0704$，整体加速约 7.04%。
算术指令性能提高 10 倍，CPI 变为 0.1：
新周期 = $5 \times 10^8 \times 0.1 + 30 \times 10^8 + 3 \times 10^8 = 0.5 \times 10^8 + 33 \times 10^8 = 33.5 \times 10^8$
加速比 = $38/33.5 \approx 1.1343$，整体加速约 13.43%。

2.40

假设一给定程序共执行了70%的算术指令、10%的取数/存数指令和20%的分支指令。

2.40.1 **[5] <2.21>** 假设执行一条算术指令、取数/存数指令和分支指令分别需要2个周期、6个周期和3个周期，求平均CPI。

$$\text{CPI}_{\text{avg}} = 0.7 \times 2 + 0.1 \times 6 + 0.2 \times 3 = 1.4 + 0.6 + 0.6 = 2.6$$

2.40.2 **[5] <1.6,2.13>** 在取数/存数指令和分支指令执行时间不变的情况下，如果要使性能提升25%，则算术运算指令的平均执行时间应该为多少?

性能提升 25% 意味着执行时间变为原来的 $1/1.25 = 0.8$ 倍。
设新的算术 CPI 为 x，其他不变。原时间 $T \propto 2.6$，新时间 $T' \propto 0.7x + 0.1 \times 6 + 0.2 \times 3 = 0.7x + 0.6 + 0.6 = 0.7x + 1.2$。
要求 $0.7x + 1.2 = 0.8 \times 2.6 = 2.08$
$0.7x = 0.88 \Rightarrow x = 1.257$
算术指令平均执行时间需降至约 1.257 周期。

2.40.3 **[5] <1.6,2.13>** 在取数/存数指令和分支指令执行时间不变的情况下，如果要使性能提升50%，则算术运算指令的平均执行时间应该为多少?

性能提升 50% 意味着执行时间变为原来的 $1/1.5 \approx 0.6667$ 倍。
$0.7x + 1.2 = 0.6667 \times 2.6 \approx 1.7333$
$0.7x = 0.5333 \Rightarrow x \approx 0.762$
算术指令平均执行时间需降至约 0.762 周期。